

Part 3, Task 7 — Responsible and Sustainable AI Report

Bias and Fairness

Known sampling and coverage limitations in the data

- **Uneven hourly coverage across years.** The Part 1 SQL analysis found recorded-hour counts ranging from ~2,100 to ~8,700 per year across 2012-2017, and Task 3's deep-learning work found the same gaps mean lag/sequence features occasionally span a real multi-hour gap while being treated as consecutive hours. Any model trained on this data has effectively seen more examples of the years/seasons with denser coverage (2016-2017) than the sparser ones (2014-2015), its performance is not uniformly reliable across all time periods, even though it was trained on "the whole dataset."
- **Single corridor, single direction.** All data is from **westbound** I-94 near Minneapolis-St Paul. Nothing here generalises to eastbound traffic, other corridors, other cities, or other climates, a model "trained on real-world data" can still be highly non-representative of the population it might get applied to if reused elsewhere without re-validation.
- **Rare-condition underrepresentation.** Severe weather categories (Squall: 4 rows, Smoke: 20 rows out of 40,575) are extremely rare in this dataset. Any model's behaviour on those conditions is trained on very little evidence and should be trusted far less than its behaviour on common conditions (Clear, Clouds), even though the model may state a prediction with the same apparent confidence either way.

Proxy labels used and their risks

- ``congestion_category`` (quartile-based) and ``high_risk`` (the proxy accident-risk label required because no real accident dataset was provided) are both **derived from traffic volume and weather alone**, not from any observed accident outcome. This is stated explicitly wherever the label is used (`scripts/common.py`, Task 1, Task 3), but the risk is real: **``high_risk`` measures "conditions historically associated with congestion + bad weather," not accident likelihood.** A city that treated this model's output as an actual crash-risk forecast could misallocate real safety resources (e.g. police presence, salt trucks) based on a correlational proxy rather than genuine incident history. **Before any operational use, this proxy must be validated against real incident data** (e.g. from the state DOT's crash records) or clearly labelled to end users as a congestion/weather-severity indicator, not an accident predictor.
- Because the label is **defined** using `is_severe_weather` and `is_low_visibility`, any classifier trained on it will necessarily learn "bad weather → high risk" almost tautologically (confirmed by the very high ROC-AUC in Task 1, 0.994-0.997). This is expected given the label construction, but it also means **the model cannot be validated as a genuine accident predictor from this data alone**, high classification performance here should not be over-interpreted as evidence the model predicts real accidents well.

How errors might be distributed unevenly

- **Rare weather conditions** (Squall, Smoke, Thunderstorm) have few training examples, so regression/classification error is very likely **higher for these conditions** than for Clear/Clouds, even though aggregate metrics (MAE, accuracy) reported in Tasks 1 and 3 are dominated by the common conditions and can mask this.
- **Holidays** (61 rows total across the whole dataset) are similarly rare; a model may systematically under- or over-predict traffic on holidays specifically because it has seen so few of them.
- **The chronological train/test split** (deliberately used to avoid leakage — see Task 1) means the test period is the **most recent** ~20% of data (Oct 2017-Sep 2018). If any structural change happened in that window that wasn't present earlier, error would concentrate there and look like a general model weakness rather than a

novelty the model genuinely couldn't have learned from older data, the Task 6 monitoring simulation's error-drift check exists specifically to catch this kind of pattern in production.

Governance and Sustainability

Oversight needed before real-world use

- **Human-in-the-loop, not autonomous action.** Given the proxy-label risk above, model outputs (risk scores, volume predictions, travel-window recommendations) should inform a human decision-maker (traffic engineer, city planner) rather than trigger automated action (e.g. automatic variable speed-limit changes) without review, at least until validated against real outcome data.
- **Periodic revalidation, not "train once."** The Task 6 monitoring simulation demonstrates *how* drift could be detected, but a real deployment needs this running on an actual schedule with a named owner responsible for acting on ALERT status, monitoring code that nobody reads is not governance.
- **Documentation and transparency.** Anyone relying on this system's outputs (e.g. a public-facing "expect high congestion risk today" advisory) should be told the risk label is a proxy, not a validated accident prediction, misrepresenting a correlational congestion/weather indicator as an accident forecast to the public would be a governance failure.
- **Equity review.** Before using travel-time recommendations to influence infrastructure or policy decisions, the city should check whether optimising for this single corridor's average commuter shifts burden onto other roads or communities not represented in this dataset at all.

Environmental and resource trade-offs

- **Model complexity vs. accuracy gain.** Task 3/6 found the lag-feature Random Forest ($R^2=0.98$) slightly *out-performed* the LSTM ($R^2=0.97$) on this dataset while being far cheaper to train (seconds vs. over a minute of CPU time here, and the gap would widen substantially on GPU-scale deep learning in a real production setting with more data/parameters). This is a concrete instance of a broader sustainability principle: **the most sophisticated model is not automatically the responsible choice**, a simpler, nearly-as-accurate model that is cheaper to train, retrain, and serve has a real environmental and cost advantage, and `mlops/model_versioning.md` documents this trade-off explicitly for whoever picks the production model.
- **Experiment tracking reduces waste.** MLflow (Task 4) means past runs don't need to be blindly re-run "just to check" what a configuration did, a small but real reduction in redundant compute over the life of a project.
- **Retraining cadence.** Because this corridor's traffic is dominated by a stable, recurring weekly pattern (Parts 1-3 consistently confirm this), it likely does **not** need frequent retraining to stay accurate, an unnecessarily aggressive retraining schedule (e.g. nightly) would consume compute for no real accuracy benefit; the monitoring simulation's error-drift check is intended to justify retraining by evidence rather than by a fixed calendar.